

WISHKIT · 통합 위시리스트

앱 화면 기능과 Firestore 구현 설명서

wishkit · 내 친구 · 살까말까 · 마이페이지
Firebase Auth · Cloud Firestore · Storage · Cloud Functions

학교 프로젝트 문서 · 코드 기준 2026년 8월

목차

1. 앱이 하는 일과 전체 구조
2. Firestore에 무엇이 어디에 있나
3. wishkit 페이지 — 리스트와 공개/비공개
4. 내 친구 페이지 — 팔로우, 위시리스트, 살까말까, 리뷰
5. 살까말까 페이지
6. 마이페이지
7. 알림이 가는 길
8. 권한 규칙이 화면과 어떻게 맞물리는지

1. 앱이 하는 일과 전체 구조

wishkit은 쇼핑몰 링크를 위시리스트에 담고, 리스트를 공개하거나 숨기고, 친구를 팔로우한 뒤 그 사람의 공개 리스트·리뷰·살까말까를 보는 Flutter 앱입니다. 계정은 Firebase Authentication(이메일/비밀번호)으로 만들고, 실제 데이터는 팀 공통 프로젝트 softstudio-wishlist-app의 Cloud Firestore에 저장됩니다. 그래서 같은 앱을 쓰는 팀원 계정은 한 데이터베이스를 공유합니다.

화면은 아래 네 개의 하단 탭으로 나뉩니다. 이 문서도 그 순서를 따릅니다.

- wishkit — 내 위시리스트. 여러 폴더(리스트)와 상품.
- 내 친구 — 팔로잉 / 팔로워 / 친구 공개 위시리스트 / 받은·보낸 살까말까 / 리뷰 피드.
- 살까말까 — 고민 중인 상품을 바구니에 모아 친구에게 보내거나 링크로 공유.
- 마이페이지 — 프로필, 내 폴더 바로가기, 리뷰, 보낸 살까말까, 알림 설정, 계정.

데이터가 움직이는 총

- 화면 — lib/screens/ 아래 각 페이지.
- 상태 — AppStore가 로그인 사용자, 탭, 상품, 친구, 알림을 메모리에 들고 화면을 갱신합니다.
- 저장소 — AccountRepository가 Firestore / Storage / Auth에 읽고 씁니다.
- 규칙 — firestore.rules가 “누가 무엇을 읽어도 되는지”를 서버에서 막습니다. 앱에서 가리는 것과 별개입니다.

2. Firestore에 무엇이 어디에 있나

최상위 컬렉션은 두 개입니다. users는 사람마다 문서 하나이고, 그 아래에 리스트·상품·팔로우·알림 같은 하위 컬렉션이 붙습니다. handles는 @아이디 중복을 막기 위한 별도 표입니다.

경로 / 키	역할	읽기 권한
users/ {uid}	프로필(이름, 이메일, @아이디, 아바타, 팔로워/팔로잉 수, FCM 토큰)	누구나 읽기
users/ {uid}/tabs/ {tabId}	위시리스트 폴더. 이름, 색, isPublic	본인 전부 / 남은 공개만
users/ {uid}/products/ {productId}	상품. listId로 폴더에 속함. isPublic을 폴더에서 복사	본인 전부 / 남은 공개만
users/ {uid}/following/ {otherUid}	내가 팔로우하는 사람	로그인한 사람
users/ {uid}/followers/ {followerId}	나를 팔로우하는 사람	로그인한 사람
users/ {uid}/notifications/ {id}	받은 알림함 (팔로우, 살까말까)	본인만
users/ {uid}/receivedBaskets/ {id}	친구가 보낸 살까말까 복사본	본인만 읽기 / 보낸 사람이 생성
users/ {uid}/sentBaskets/ {id}	내가 보낸·링크 공유한 살까말까	본인만
users/ {uid}/reviews/ {reviewId}	블로그형 상품 리뷰	로그인한 사람 읽기 / 본인만 쓰기
handles/ {handleId}	@아이디 → uid 매핑. 중복 방지·아이디로 계정 찾기	누구나 읽기

사진 파일은 Firestore가 아니라 Storage에 있습니다. 프로필은 `avatars/{uid}/...`, 리뷰 사진은 `reviews/{uid}/{reviewId}/...` 입니다. 살까말까 바꾸니는 기기 안에 `SharedPreferences`로도 잠시 들고 있다가, 친구에게 보내는 순간에만 상대 `receivedBaskets`로 복사됩니다.

계정 한 명의 대략적인 모양

```
users/abc123
name, handle, email, avatarUrl, followers, following, fcmTokens
tabs/all → { id: all, name: 전체, isPublic: true }
tabs/list-... → { name: 여름옷, isPublic: false, color: #... }
products/1 → { listId: list-..., name: ..., isPublic: false }
following/친구uid
followers/다른uid
```

3. wishkit 페이지 — 리스트와 공개/비공개

첫 탭입니다. 위에서 리스트(폴더)를 고르고, 그 안에 담은 상품을 봅니다. 고정 탭 전체는 모든 상품을 한눈에 보는 뷰이고, 그 외 탭이 실제 폴더입니다. 새 계정은 '전체'만 만들어 두고, 데모 카테고리에는 넣지 않습니다.

화면에서 할 수 있는 일

- 리스트 추가 — 탭 줄의 + . 이름과 함께 공개 여부를 고를 수 있습니다. 기본은 비공개입니다.
- 더블탭 — 리스트 이름 바꾸기, 삭제, 공개/비공개 전환.
- 꺾 눌러 드래그 — 탭 순서 바꾸기. 아이템을 다른 탭 위로 드래그하면 그 리스트로 이동합니다.
- 공개/비공개 배너 — '전체'가 아닌 리스트 상단에 자물쇠/지구본이 있습니다. "친구들이 이 리스트를 볼 수 있어요" / "나만 볼 수 있어요".
- 상품 담기 — 오른쪽 아래 '공유 담기'. 쇼핑몰 URL을 붙여 넣거나, 다른 앱에서 링크로 공유해 열면 파서가 이름·가격·이미지를 읽습니다.
- 상품 카드 — 눌러 상세. 삭제, 살까말까 바꾸니에 넣기, (내 상품이면) 메모·리뷰 쓰기.

Firestore에 어떻게 붙나

리스트 하나 = `users/{내uid}/tabs/{tabId}` 문서. 필드에는 `id`, `name`, `isPublic`, `color`가 들어갑니다. 상품 하나 = `users/{내uid}/products/{상품id}`. 상품은 `listId`로 어느 탭에 속하는지 알고, 같은 문서에 `isPublic`도 같이 저장합니다.

왜 상품에도 `isPublic`이 있냐면, Firestore 규칙은 쿼리를 필터처럼 써 주지 않기 때문입니다. 친구가 "이 사람 상품 전부"를 요청하면 비공개 문서가 끼어 규칙이 쿼리 전체를 거절합니다. 그래서 친구 쪽은 `isPublic == true` 인 것만 요청하고, 서버도 그 문서만 내줍니다. 리스트 공개를 바꾸거나 상품을 다른 리스트로 옮기면, 앱이 상품 쪽 `isPublic`을 같이 맞춥니다.

- 내 앱이 켤 때: `loadTabs` / `loadProducts` 로 내 하위 컬렉션 전체를 읽습니다. 본인이라 비공유도 보입니다.
- 저장: `saveTabs`, `upsertProduct`. 리스트를 저장할 때 소속 상품의 `isPublic`도 맞춥니다.
- 상품 URL: `ParsingBridge`가 파싱 엔진 또는 웹뷰 스크립트로 `ParsedProductInfo`를 만들고 `addParsedProduct`가 Firestore에 set 합니다.

공개와 비공개에 실제 의미

비공개 리스트는 내 wishkit과 마이페이지 폴더에는 그대로 있습니다. 친구의 'wishlist' 탭에는 안 나옵니다. 규칙이 배포된 뒤에는 콘솔이나 직접 쿼리로도 남의 비공개 문서를 읽을 수 없습니다. 살까말까로 내가 골라 보낸 상품은 예외입니다. 그때는 위시리스트를 열어 주는 게 아니라, 보낸 순간의 상품 스냅샷을 상대방 받은함에 복사합니다.

4. 내 친구 페이지

두 번째 탭입니다. 위쪽에 이름·아이디 검색, 새로고침, 알림 종 아이콘이 있고, 그 아래 다섯 개 칩이 있습니다. 팔로잉 · 팔로워 · wishlist · 살까말까 · 리뷰.

4-1. 팔로잉

내가 팔로우 중인 사람과, 아직 안 한 사람(친구 찾아보기)이 같이 나옵니다. 검색하면 이름·@아이디로 걸러집니다. 팔로우 버튼을 누르면 관계가 생기고, 다시 누르면 취소됩니다.

구현

A가 B를 팔로우하면 한 번의 batch로 네 곳이 바뀝니다. A의 following에 B 문서, B의 followers에 A 문서, A.following +1, B.followers +1. 언팔로우는 그 반대입니다. 자기 자신을 팔로우하는 건 무시합니다.

팔로우가 쓰는 경로

```
users/{나}/following/{상대uid} 생성 또는 삭제
users/{상대}/followers/{나uid} 생성 또는 삭제
users/{나}.following ± 1 (규칙: 카운트만 한 칸)
users/{상대}.followers ± 1
users/{상대}/notifications/{새id} type: follow (팔로우할 때만)
```

친구 목록 자체는 users 컬렉션을 최대 80명까지 읽은 뒤, 내가 following에 넣은 uid와 맞춰 isFollowing을 표시합니다. 카드에 보이는 '위시리스트 n · 아이템 n'은 그 사람의 공개 탭/상품만 셉니다.

4-2. 팔로워

나를 팔로우한 사람 목록입니다. 인스타그램처럼 삭제를 누르면 그 사람의 팔로잉에서 내가 빠지고, 내 팔로워에서도 그 사람이 빠집니다. 상대는 더 이상 나를 팔로우하지 않은 상태가 됩니다.

구현은 removeFollower 입니다. 내 followers/{상대}와 상대 following/{나}를 지우고 양쪽 카운트를 내립니다. 마이페이지에서 팔로워 숫자를 눌러 들어가는 목록도 같은 following / followers 하위 컬렉션을 읽습니다.

4-3. wishlist (친구 위시리스트 공유)

내가 팔로우하는 사람의 공개 리스트만 카드로 보여 줍니다. 비공개 리스트와 '전체' 탭은 빼 둡니다. 카드를 열면 그 리스트에 속한 공개 상품이 나옵니다. 이것이 앱에서 말하는 위시리스트 공유입니다. 링크를 따로 만드는 기능이 아니라, 팔로우 관계 + 공개 플래그로 피드에 뜨는 방식입니다.

구현

loadFriendWishlists가 팔로잉 친구마다 tabs.where(isPublic == true), products.where(isPublic == true) 를 요청합니다. 예전처럼 전부 가져온 뒤 앱에서 가리지 않습니다. 규칙이 배포되면 그 쿼리만 통과하고, 비공개

문서는 서버가 막습니다.

4-4. 살까말까 (친구 탭 안의 피드)

여기 있는 살까말까는 보낸 것 + 받은 것을 시간순으로 모아 보여 주는 피드입니다. 바구니는 세 번째 하단 탭에서 만들고, 결과는 이 탭과 마이페이지 '내가 보낸 살까말까'에도 쌓입니다. 받은 카드를 누르면 상대가 고른 상품 목록을 위시리스트처럼 봅니다.

받은 쪽은 `users/{나}/receivedBaskets` 를 실시간으로 구독합니다. 보낸 쪽은 `users/{나}/sentBaskets` 와 기기의 `SharedPreferences`를 같이 씁니다.

4-5. 리뷰

내가 팔로우하는 사람이 쓴 블로그형 상품 후기 피드입니다. 제목, 본문, 별점(mood), 사진, 어떤 상품에 대한 글인지가 들어갑니다. 리뷰 탭이 켜져 있을 때만 오른쪽 아래 '리뷰 쓰기' 버튼이 나옵니다. 내 위시리스트 상품 상세에서도 '이 상품 리뷰 쓰기'로 들어올 수 있습니다.

구현

글은 `users/{작성자uid}/reviews/{reviewId}` 에 저장합니다. 사진은 `Storage reviews/{uid}/{reviewId}/n.jpg` 에 올리고 다운로드 URL을 문서에 넣습니다. 피드는 팔로잉 친구마다 `loadReviews`를 호출해 모은 뒤 최신순으로 정렬합니다. 지금은 로그인한 사용자라면 남의 리뷰 컬렉션을 읽을 수 있습니다. 위시리스트처럼 `isPublic`을 리뷰에 두지는 않았습니다. 리뷰를 올려도 팔로워에게 푸시 알림을 보내지는 않습니다.

알림 중

친구 페이지 오른쪽 종을 누르면 알림함입니다. 누군가 나를 팔로우했거나, 살까말까를 보냈을 때만 쌓입니다. 리뷰 작성과 리스트 공개는 알림을 만들지 않습니다. 알림함은 `users/{나}/notifications` 스냅샷입니다. 열면 읽음 처리됩니다.

5. 살까말까 페이지

세 번째 하단 탭입니다. 위시리스트와 별개인 고민 바구니입니다. 살지 말지 친구 의견을 묻고 싶은 상품을 여기에 모았다가, 고른 것만 공유합니다. 위시리스트를 통째로 공개하는 것과 목적이 다릅니다.

화면 구성

- 위에 담긴 상품 목록. 체크박스 이번엔 공유할 것만 고릅니다.
- 상품 추가하기 — 내 위시리스트 폴더를 골라 그 안 상품을 바구니에 넣습니다. 이미 담긴 것은 건너뜁니다.
- 상품 상세의 '바구니에 추가'로도 넣을 수 있습니다.
- 공유하기 — 체크한 개수가 1개 이상일 때만 열립니다.

공유 방식 세 가지

1) 앱 친구에게 보내기

팔로잉 중인 친구를 여러 명 고릅니다. 보내기를 누르면 각 친구의 receivedBaskets에 상품 목록이 통째로 복사되고, 그 친구 알림함에 type: basket 이 생깁니다. 내 sentBaskets에도 같은 내용이 남고, 마이페이지 '내가 보낸 살까말까'에서 다시 보낼 수 있습니다.

앱으로 보낼 때

```
보내는 사람 users/{나}/sentBaskets/{id}
items: [ 상품 스냅샷... ], fromUid, recipientUids
받는 사람 users/{상대}/receivedBaskets/{새id}
같은 items 복사. 상대만 읽기 가능
알림 users/{상대}/notifications/{새id} type: basket
로컬 SharedPreferences 의 shared_baskets (미리보기·재전송용)
```

상대는 내 tabs/products를 읽지 않습니다. 그래서 비공개 리스트에 있던 상품도 내가 골라 보내면 그 복사본은 볼 수 있습니다. 위시리스트 공개 설정과 독립입니다.

2) 링크 복사

선택한 상품으로 SharedBasket을 만들고 /shared/{id} 형태의 주소를 클립보드에 넣습니다. 앱 안에서 그 주소를 열면 위시리스트처럼 미리보기가 됩니다. 이 링크 바구니도 sentBaskets와 로컬 저장소에 남습니다.

3) 카카오톡

현재 이 브랜치 화면에는 '카카오 공유는 추후 연동 예정' 안내가 떠 있습니다. 앱 친구 전송·링크 복사와 달리, 카카오 SDK로 실제로 보내지는 상태는 아닙니다.

바구니가 Firestore에 바로 안 가는 이유

담기·체크·빼기는 우선 기기 안(SharedPreferences, 키 basket_items)에서만 일어납니다. 고민 중인 목록을 서버에 올리지 않기 위해서입니다. 공유 버튼을 눌렀을 때만 서버에 스냅샷이 생깁니다.

6. 마이페이지

네 번째 하단 탭입니다. 나에 대한 요약과 설정, 내 콘텐츠로 들어가는 허브입니다.

프로필 카드

- 아바타, 이름, @아이디(handle).
- 팔로워 n — 누르면 팔로워 목록. Firestore users/{나}/followers.
- 팔로잉 n — 누르면 내가 팔로우하는 사람. users/{나}/following.
- 아이템 n — 내 위시리스트 상품 수(비공개 포함, 내 것이므로 전부).
- 리뷰 n — 누르면 내가 쓴 리뷰 목록 /my-reviews.

내 wishlist 폴더

'전체'를 제외한 내 리스트가 색 칩으로 깔립니다. 칩을 누르면 wishkit 탭으로 가며 그 리스트가 선택됩니다. + 로 폴더만 빠르게 추가할 수도 있습니다. 공개/비공개 토글은 wishkit 쪽 배너·더블탭에서 합니다. 데이터는 같은 tabs 컬렉션입니다.

버튼 모음

내 리뷰 — 내가 쓴 글 목록. 수정·삭제는 작성자만. Firestore `users/{나}/reviews`.

내가 보낸 살까말까 — 앱 친구에게 보냈거나 링크로 만든 바구니 기록. 다시 다른 친구에게 재전송할 수 있습니다. `sentBaskets` + 로컬 `shared_baskets`.

알림 설정 — ‘새 팔로우’, ‘살까말까 장바구니’를 알림함에 넣을지 고릅니다. 이 스위치는 Firestore가 아니라 기기 `SharedPreferences`에 저장됩니다. 서버가 알림 문서를 만드는 것 자체는 막지 않고, 내 화면에서 숨길지만 정합니다. 알림함 열기 버튼으로 친구 페이지와 같은 목록으로 갑니다.

앱 정보 — 이름·버전 다이얼로그.

오른쪽 위 톱니 — 계정 설정

- 프로필 수정 — 이름, 아바타(갤러리 업로드 또는 프리셋). `Storage avatars/{uid}/...`에 올린 뒤 `users/{uid}.avatarUrl` 갱신. @아이디를 바꾸면 `handles` 문서도 같이 옮깁니다.
- 비밀번호 수정 — Firebase Auth `updatePassword`. Firestore 문서는 안 바꿉니다.
- 로그아웃 — Auth 세션 종료, 기기 상태 비움.
- 탈퇴하기 — 비밀번호 재확인 후 Auth 계정 삭제. 규칙이 허용하는 범위에서 내 `users` 문서·하위 컬렉션·handle을 지웁니다. 위시리스트도 함께 사라집니다.

로그인·회원가입은 마이페이지 밖(첫 실행)이지만 같은 계정 시스템을 씁니다. 가입 시 Auth만 만들고 이메일 인증 후 `users/{uid}`와 `handles/{아이디}`를 만듭니다. 아이디로 가린 이메일을 찾아 비밀번호를 재설정하는 복구 화면은 `handles`와 공개 프로필 읽기를 이용합니다.

7. 알림이 가는 길

알림은 “누가 내 문서를 읽었는지”가 아니라, 상대 받은함에 문서를 써 주는 방식입니다. A가 B를 팔로우하면 알림은 B의 `notifications`에 생깁니다. A의 알림함이 아닙니다.

- 앱이 켜져 있을 때 — `notifications`와 `receivedBaskets` 스냅샷을 듣다가 새 문서가 오면 로컬 배너를 띄웁니다 (`flutter_local_notifications`).
- 앱이 꺼져 있을 때 — Cloud Function `pushOnInbox`가 `notifications` 생성 이벤트를 보고, 그 사용자 `users/{uid}.fcmTokens`로 FCM을 보냅니다. 리뷰/리스트 타입은 함수가 무시합니다.
- 토큰은 로그인 후 기기가 프로필 문서의 `fcmTokens` 배열에 넣습니다.

알림 설정 스위치는 내 화면 필터입니다. 꺼 두어도 상대가 팔로우하면 서버 문서는 생길 수 있고, 내 알림함 목록에서 해당 종류만 안 보이게 할 수 있습니다.

8. 권한 규칙이 화면과 어떻게 맞물리는지

앱 UI만으로 가리는 것과, Firestore가 거절하는 것은 다릅니다. 아래는 현재 코드(PR의 규칙) 기준입니다. 콘솔에 게시한 뒤에야 서버가 이렇게 막습니다.

경로 / 키	역할	읽기 권한
프로필 <code>users/{uid}</code>	친구 검색, 아이디 복구	로그인 없이 읽기 가능
<code>tabs / products</code>	위시리스트	본인 전부. 남은 <code>isPublic==true</code> 만
<code>following / followers</code>	관계	로그인한 사람 읽기. 쓰기는 당사자

notifications	알림함	본인 읽기. 생성은 fromUid == 나
receivedBaskets	받은 살까말까	본인 읽기. 생성은 fromUid == 나
sentBaskets	보낸 살까말까	본인만
reviews	리뷰	로그인한 사람 읽기. 쓰기는 본인
handles	@아이디	누구나 읽기. 쓰기는 자기 uid

프로필이 공개 읽기인 이유는 친구 찾아보기와, @아이디로 가려진 이메일을 찾아 비밀번호를 재설정하기 위해서입니다. 이메일이 프로필 문서에 있으면 그 값도 읽힐 수 있습니다. 학교 프로젝트에서는 이렇게 두었고, 나중에 서비스를 열 때는 필드를 줄이거나 복구 전용 클라우드 함수로 옮기는 편이 안전합니다.

관련 코드 위치 (참고)

주요 파일

화면 lib/screens/wishlist/wishlist_screen.dart
 lib/screens/friends/friends_screen.dart
 lib/screens/salkamalka/salkamalka_screen.dart
 lib/screens/mypage/mypage_screen.dart
 상태 lib/data/app_store.dart
 저장 lib/services/account_repository.dart
 모델 lib/models/models.dart
 규칙 wishlist-appversion2/firestore.rules
 스토리지 wishlist-appversion2/storage.rules
 푸시 wishlist-appversion2/functions/index.js

이 문서는 앱 코드와 규칙을 기준으로 기능을 설명합니다. Firebase 콘솔 규칙이 아직 예전(로그인하면 tabs/products 전부 읽기)이면, 서버는 비공개 리스트도 내줄 수 있습니다. 앱 최신본과 규칙 게시를 맞춘 뒤에 비공개의 의미가 화면과 데이터베이스에서 같아집니다.